

Github link

[Link to github](#)

Commits before "Completed 2.1" might be deleted and overwritten due to SSH/GPG key changes :(.

Part 1: Threads and Processes

Process and threads are essential data-structures for computer systems to queue and safely execute computer instructions. **Processes execute a program with a restricted rights, overseen by the kernel. Threads are independent and separate sequences that execute instructions.**

The **main differences** between the two is flexibility and access to system resources. Processes do not necessarily share the same memory and share resources (can be done with Shared Memory) but threads in the same process usually do. This allows for smoother and faster communication. The second difference is that threads themselves have their own registers, stack and state but still rely on the process for system calls. This means process are more "heavyweight", needing proper time and execution to run and terminate. Thread are depended on the parent process for those kinds of resources making it more "lightweight".

Most operating systems used a combination of threads and processes to manage complexity, for example in "Kernel Threads" where the operating system allows multiple process to run threads. Basically those threads have the same resources as the process they are running on.

- **In some cases threads are preferred to achieve responsiveness and background work.** This is done by running a parallel thread in the background. For example; When downloading a movie, the option to cancel should work. Here the cancel option is the parallel thread that handles input. A parallel process might be too slow and unnecessary use of system resources just to say "no".
- **Process can be used for heavier system services.** Important services such as in backend server or OS daemons are usually use process. This is to securely and strictly contain a service on interruptes or failures.

For threads to work properly and to communicate effectively, most threads have a TCB, Thread Control Block. The operating system need information on the threads state, so TCB makes it into a object of some sort. The TCB contains the computation being performed by the thread (stack and registers) and the metadata about the thread . Stack and register data is to **make sure that the OS/thread schedulers know each others states and so that a thread can properly resume, suspend or stop**. Metadata for identification ***for accurate suspension and resuming***. This component is also essential for thread context switching.

For a thread context switch to occur, a voluntary call into the thread library or an involuntary interrupt or exeption need to happen. On a voluntary call, its usually means that the thread gave up a processor, giving it to a different thread. While an involuntary one happens involuntarily and the system needs to decide to continue by restoring or run a different thread by saving and letting it go to the handler.

Part 2: C program with POSIX Threads.

Given:

```
#include <stdio.h>
#include <pthread.h>
#define NTHREADS 10
pthread_t threads[NTHREADS];
void *go (void *n) {
    printf("Hello from thread %ld\n", (long)n);
    pthread_exit(100 + n);
    // REACHED?
}

int main() {
    long i;
    for (i = 0; i < NTHREADS; i++) pthread_create(&threads[i], NULL, go,
(void*)i);
    for (i = 0; i < NTHREADS; i++) {
        long exitValue;
        pthread_join(threads[i], (void*)&exitValue);
        printf("Thread %ld returned with %ld\n", i, exitValue);
    }
    printf("Main thread done.\n");
    return 0;
}
```

Function `*go()` is run everytime a new thread is created. This can be idenfited in the syntax for `pthread_create`. Where the third paramater is the starting function. In this

use case, the function `*go` gets passed an argument `n` which is the iteration-number in the for-loop `i`, prints it out with a message, and exits with its TID + 100.

When running the code multiple times you may notice that the order of "Hello from thread X" is different each time. This is due to the properties of multithreading. When making a thread, the scheduler determines when it gets CPU time. Some threads need to wait for something else to finally run. Therefore a thread 0 does not guarantee it will run before a thread 1.

When counting amount of threads you need to also include the main thread is also included. For example, in this code, when thread 8 is running you have a minimum of 2 threads running and maximum of 11 (thread 0 to and including 9 is 10 threads + main thread)

`pthread_join()` function waits for a specified thread terminate. It does not do anything else until target thread finishes. Changing the `*go()` function to:

```
void *go (void *n) {
    printf("Hello from thread %ld\n", (long)n);
    // Sleep if Thread ID = 5.
    if (n==5){
        sleep(2);
    }
    pthread_exit(100 + n);
    // REACHED?
}
```

-Will cause the whole thread 5 to wait, but due to `pthread_join(threads[i], (void*)&exitValue);`, the prints won't happen until sleep is finished. This does not mean the threads are pausing, they are still running. So it seems that the whole system is waiting for thread 5

When returning, it returns for thread X. But in the function for `*go()`, it has exited. So the return is that the thread has exited.